

Lesson 2 — Events, HTTP, and Express

Node.js/Express

Game Plan

- Warm-Up
- Event Emitters & Listeners
- Raw Node HTTP
- Postman and POST requests
- Express basics
- Routes and controllers
- Core vs advanced assignment split
- Wrap-Up

Warm-Up (5 min)

In chat or out loud:

1. What is one difference between Node and browser JavaScript?
2. What is one thing from Week 1 that still feels fuzzy?

Event Emitters

Node has a built-in `EventEmitter` class.

```
const EventEmitter = require("events");
const emitter = new EventEmitter();

emitter.on("time", (message) => {
  console.log("Time received:", message);
});

emitter.emit("time", new Date().toString());
```

`on()` listens. `emit()` announces.

Why Event Emitters?

They let one part of a program notify other parts.

- The event has a name
- One event can have one or more listeners
- Listeners run when the event is emitted

Mentor prompt:

Where have you seen this pattern in browser JavaScript?

Raw Node HTTP

Node's built-in `http` module can create a server.

```
const http = require("http");

const server = http.createServer((req, res) => {
  res.writeHead(200, { "Content-Type": "application/json" });
  res.end(JSON.stringify({ time: new Date().toString() }));
});

server.listen(8000);
```

Stop it with `Ctrl-C`.

HTTP Requests Have Parts

When a request arrives, `req` gives you:

- `req.method` — `GET`, `POST`, `PATCH`, `DELETE`
- `req.url` — path such as `/time`
- `req.headers` — metadata such as `Content-Type`
- `body` — arrives in chunks and must be read

Headers are metadata.

`Content-Type` says what kind of body is being sent or returned.

Assignment 2 Raw Routes

Students build `assignment2/sampleHTTP.js` .

Core routes:

- `GET /time` returns JSON with a `time` property
- `GET /timePage` returns HTML with a button
- `POST /echo` returns the JSON body under `weReceived`

Advanced routes:

- Unknown raw HTTP route returns `404`
- Invalid JSON to `POST /echo` returns `400`

Reading the Body (Raw HTTP)

The request body arrives in chunks.

```
let body = "";

req.on("data", (chunk) => {
  body += chunk;
});

req.on("end", () => {
  const parsedBody = JSON.parse(body);
  res.end(JSON.stringify({ weReceived: parsedBody }));
});
```

This is exactly the kind of work Express helps with.

Testing POST Requests with Postman

Browsers naturally make `GET` requests from the address bar.

For `POST /echo`, use Postman:

1. Method: `POST`
2. URL: `http://localhost:8000/echo`
3. Body: raw JSON
4. Send:

```
{  
  "message": "Hello from Postman"  
}
```

HTTP vs Express

	Node <code>http</code>	Express
Routing	Manual <code>if</code> checks	<code>app.get()</code> , <code>app.post()</code>
JSON body	Manual chunks	<code>express.json()</code>
Responses	<code>res.writeHead()</code> , <code>res.end()</code>	<code>res.status().json()</code>
Organization	One file gets messy	routes + controllers

Hello Express

```
const express = require("express");

const app = express();

app.use(express.json());

app.get("/", (req, res) => {
  res.send("Hello, World!");
});

app.post("/testpost", (req, res) => {
  res.status(200).json({ message: "POST route works" });
});
```

Export `app` and `server`

Assignment 2 asks for:

```
const port = process.env.PORT || 3000;

const server = app.listen(port, () => {
  console.log(`Server is listening on port ${port}...`);
});

module.exports = { app, server };
```

Why?

Supertest needs `app`, and tests need to close `server`.

Routes and Controllers

Students organize Express code:

```
controllers/  
  timeController.js  
routes/  
  timeRoutes.js
```

Controller:

```
function getTime(req, res) {  
  res.status(200).json({ time: new Date().toString() });  
}
```

Route:

```
router.get("/time", timeController.getTime);
```

Mounting a Router

In `app.js` :

```
const timeRouter = require("./routes/timeRoutes");  
app.use("/api", timeRouter);
```

That means:

- `router.get("/time", ...)`
- becomes `GET /api/time`

And:

- `router.post("/echo", ...)`
- becomes `POST /api/echo`

Advanced: Unknown Routes

Express fallback route:

```
app.all("*", (req, res) => {  
  res.status(404).json({  
    message: `No route found for ${req.method} ${req.path}`,  
  });  
});
```

Mentor note:

Do not over-teach middleware here. Lesson 3 covers it deeply.

Advanced: Server Lifecycle Polish

Useful but not required by TDD:

```
server.on("error", (err) => {  
  if (err.code === "EADDRINUSE") {  
    console.error(`Port ${port} is already in use.`);  
  }  
});
```

Also mention graceful shutdown with `SIGINT` / `SIGTERM`.

Keep this short unless students ask.

Core vs Advanced Tests

Assignment 2 is split:

Core:

```
npm run tdd assignment2a
```

Advanced:

```
npm run tdd assignment2b
```

Core must pass before submission. Advanced is optional.

Assignment Preview

Core tasks:

1. `assignment2/events.js`
2. `assignment2/sampleHTTP.js`
3. `raw GET /time`, `GET /timePage`, `POST /echo`
4. `root app.js`
5. `controllers/timeController.js`
6. `routes/timeRoutes.js`

Advanced:

1. `raw 404`
2. `invalid JSON 400`
2. `Express unknown route 404`

Wrap-Up

In chat:

1. What does `emitter.on()` do?
2. Why do raw HTTP POST bodies require `"data"` and `"end"` ?
3. What does Express make easier?
4. How does `/api` get added to `/time` ?

Confidence Check

On a scale of 1-5:

How comfortable do you feel starting Assignment 2?

Resources

- <https://nodejs.org/api/events.html>
- <https://nodejs.org/api/http.html>
- <https://expressjs.com/en/guide/routing.html>
- Postman VS Code Extension docs
- Ask questions in Slack

Closing

This week:

Events, raw HTTP, Postman, and your first Express routes.

Next week:

Middleware, error handling, status codes, and debugging Express apps.

See you then!